

Introduction to the Xe-HPG Architecture

This white paper describes the components of Xe-HPG, a new high-performance graphics architecture designed for discrete GPUs.

Contents

Overview.....	1
Xe-HPG Highlights	2
17 Teraflop SP FP32 Performance.....	2
Deep Learning Inference Support	2
Intel Xe Super Sampling (XeSS) for Gaming.....	2
Real-Time Ray Tracing	2
Mesh Shading.....	2
Variable Rate Shading.....	3
Sampler Feedback	3
Unified Lossless Compression ...	3
High-Bandwidth Device Memory	3
PCI Express Gen4	3
Xe-HPG Graphics Architecture...	4
New Xe-HPG Render Slice and Xe-core Design	4
Xe-core Overview	5
Xe-core Fixed Function.....	7
Xe-core L1\$.....	7
Xe-HPG Slice Common	7
L2 Cache, Compression, and Memory.....	8
Global Assets	8
Specifications Tables	9
Peak Rates	9
Graphics API Support.....	11
Conclusion.....	12

Overview

The Intel® Arc™ A-series GPUs —formerly code-named Alchemist—feature Xe-HPG, a new high-performance graphics architecture designed for discrete GPUs. This white paper describes the components of Xe-HPG that deliver not only traditional rasterization and compute performance, but also real-time ray tracing and mesh shading for the next generation of PC gaming visuals. The paper serves as an introduction to the Xe-HPG architecture, in support of the programming guides.

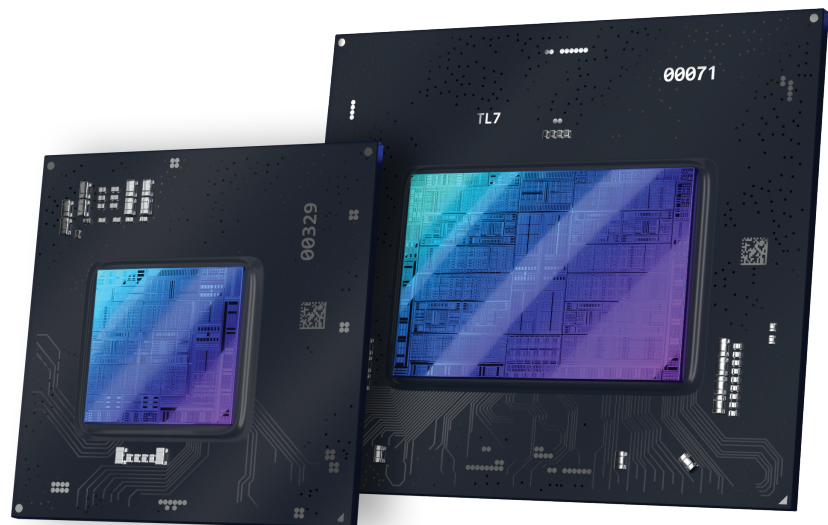


Figure 1. From the Intel Arc A-series graphics: the ACM-G10 GPU and the ACM-G11 GPU.

Xe-HPG Highlights

State-of-the-art immersive computer graphics for TV shows and movies are the current benchmark for visual effects achievement, which makes them the near-term goal for real-time gaming. This goal is driving an insatiable demand for the latest technologies and research.

Intel Xe Architecture is a proprietary Intel® GPU technology that represents a significant leap in terms of efficiency and performance. Intel's Xe-LP graphics was the first instance of this technology, pushing the performance on highly portable devices. Xe-HPG takes this baseline and springboards from it, with performance gains targeted at the discrete GPU market to meet the consumer demand for PC gaming.

Intel's Xe-HPG GPUs (such as the ACM-G10 and the ACM-G11) are built on Taiwan Semiconductor's (TSMC) N6 process. The GPUs power the new Intel Arc A-series discrete cards and mobile products (formerly code-named Alchemist, also referred to as DG2). While Xe-HPG will ship with a maximum of 32 Xe-cores, other reduced configurations match a complete family of SKUs.

The goal of the Xe-HPG design is to extend the baseline of significant architectural and micro-architectural enhancements provided by the Xe-LP architecture, and scale it up for highly performant discrete GPUs. Xe-HPG graphics architecture delivers a massive improvement in floating point and integer compute capabilities that takes advantage of a high-bandwidth memory hierarchy.

17 Teraflop SP FP32 Performance

Xe-HPG contains up to 32 Xe-cores and 512 vector engines (XVE), which increase the core compute capability per clock by greater than 5¹ over Xe-LP devices with up to 96 vector engines, previously called EUs, or execution units. Additionally, it contains dedicated matrix engines (XMX), and a real-time ray tracing fixed-function block. Xe-HPG addresses the corresponding bandwidth needs by using the latest graphics memory technology (Graphics Double Data Rate 6, or GDDR6).

Xe-HPG has made considerable improvements to the Xe-LP design, realized in gaming and compute workloads. Advances were targeted to improve XVE latency, as well as increasing latency tolerance, and improvements in the caching hierarchy.

Deep Learning Inference Support

Deep learning (DL) applications have exceeded all expectations in the last five years. The primary building blocks of many deep learning applications are high-throughput multiply-accumulate operations. These can be used to implement optimized deep learning convolutions, as well as optimized general matrix-multiply (GEMM) operations. In

addition to increasing the peak FP32 performance 5x over Xe-LP, Xe-HPG includes XMX engines built with systolic arrays to enable up to 275 teraOPS (TOPs) of compute performance. Xe-HPG supports FP16, BF16, INT8, INT4, and INT2 data formats to provide a range of quantization for inferencing workloads, which can tolerate the lower precision.

Optimized use of this new hardware is exposed through [Intel® Distribution of OpenVINO Toolkit™](#) and [Microsoft* WinML APIs](#), as well as Intel-optimized Compute Library for Deep Neural Networks (CLDNN) kernels. CLDNN is an Intel library built to expose optimized GPU implementations in the Intel® Deep Learning Deployment Toolkit and used by a variety of Intel-optimized deep learning (DL) frameworks, including Caffe*, Torch*, TensorFlow*, and Intel® Math Kernel Library for Deep Neural Networks (Intel® MKL-DNN).

Xe-HPG also supports all the DirectX* 12 Ultimate features to provide the highest visual quality without compromise, as described in the features that follow.

Xe-HPG has made considerable improvements to the Xe-LP design, realized in gaming and compute workloads. Advances were targeted to improve XVE latency, as well as increasing latency tolerance, and improvements in the caching hierarchy.

Intel Xe Super Sampling (XeSS) for Gaming

One of the applications of the XMX engines in Xe-HPG is the application of XeSS, a method targeted at upscaling content rendered at a lower resolution with up to 4x increase in resolution while maintaining excellent image quality. Most modern games and game engines use a temporal antialiasing pass, and XeSS uses the available TAA inputs to upscale the resolution up to 4x, with little-to-no image quality impact.

Real-Time Ray Tracing

Ray tracing is widely regarded as the highest fidelity technique for simulating physical light behavior in 3D graphics and is used in most offline-rendered computer-generated films. In games, ray tracing is used to enable global illumination, realistic shadows, reflections, ambient occlusion, and more. Games mandate high performance in real time and use rasterization instead of ray tracing; this enables a trade-off between quality and performance.

Current advances in the levels of performance allow for a hybrid model that enables ray-tracing effects layered alongside the traditional rasterization. Xe-HPG fully supports DirectX* ray tracing (DXR) 1.0, DXR 1.1, and Vulkan* Run Time Libraries (Vulkan*RT); this enables ISVs to use the new ray-tracing techniques in their existing gaming pipelines.

Mesh Shading

The mesh-shading feature supports a one- or two-stage compute-like shader pipeline, which is used to generate

3D primitives. These are fed directly into the existing rasterization pipeline. Mesh shading gives applications increased flexibility and performance for the definition and generation of 3D primitives. The mesh-shading capability can be used as a replacement for the 3D geometry pipeline (vertex input, vertex shading, tessellation, and geometry shading). Microsoft* introduced mesh-shading Tier 1 as a new feature in DirectX 12 Ultimate*, and an extension also exists for Vulkan.

Xe-HPG has made considerable improvements to the Xe-LP design, realized in gaming and compute workloads. Advances were targeted to improve XVE latency, as well as increasing latency tolerance, and improvements in the caching hierarchy.

Variable Rate Shading

Variable rate shading (VRS) is a technique for giving the developer better control over which pixels to spend time shading. For example, if an object or region of the screen is undergoing motion blur, or objects are hidden behind smoke or fog, it often does not make sense to spend time shading when the results are not seen by the user in the final image. Display densities have increased each generation, with double or even quadruple the number of pixels in the same region of space. This motivates the need for better control of the amount of shading across the display, so shading resources are used where they are needed the most.

While Xe-LP already supports VRS Tier 1, Xe-HPG additionally supports VRS Tier 2. VRS Tier 1 included the ability to set the shading rate for every draw call. VRS Tier 2 enables you to set the shading rate via an image mask, where each pixel of the image mask defines the shading rate for an 8x8 tile of the render target. Every tile of the image mask can be updated based on a variety of parameters that impact on the required shading resolution. For example, the amount of color intensity changes within a region of the previous frame. Motion of objects in the scene, camera parameters, or any number of other factors can vary across the display. In addition to being able to adjust the shading rate using an image mask, VRS Tier 2 also adds the ability to set the shading rate for every triangle rendered, giving the programmer the ultimate level of control.

Intel supports a finer-grained image mask than other hardware vendors at 8x8, but also an expanded range of shading rates going beyond the standard 1x1, 1x2, 2x1, and

2x2, adding the ability to use shading rates 2x4, 4x2, and 4x4. These additional shading rates give the programmer even more control of performance, with no loss in perceived visual quality—a 4x4 coarse pixel has four times fewer invocations than a 2x2 coarse pixel. This means that the same number of displayable pixels can be shaded using only 25% of the resources when compared to using 2x2 coarse pixels.

Sampler Feedback

Sampler feedback is a hardware-accelerated feature introduced in DirectX Feature Level 12_2 and supported by Xe-LP and later. Conceptually, it is the reverse of texture sampling: the `sample()` shader intrinsic reads a number of texels from a texture and returns an average value. The new `WriteSamplerFeedback()` shader intrinsic writes to a binary resource “marking” the texels that would have been read. This enables two important uses: sampler feedback streaming, and texture space shading.

This feature allows pixel shading to be independent of the geometric complexity of screen resolution and adds new texture level-of-detail methods to control the level of shading detail. These abilities are introduced to temporally decouple the shading pipeline from the traditional sample-, pixel-, or coarse-shading rates. Sampler feedback can provide sizable gains over traditional pixel-shader approaches, as shader complexity, geometry, or MSAA samples increases.

Unified Lossless Compression

Unified lossless compression is an evolution of Xe-LP’s end-to-end compression feature. Its primary purpose is to reduce read/write memory bandwidth and power. For memory-constrained workloads, it also improves overall system performance, and can be applied to all resource types. It supports seamless consumer-producer paths across all rendering, display, compute, and media units.

High-Bandwidth Device Memory

Higher bandwidth requirements are driven by newer AI/DL inference workloads. Xe-HPG supports up to 17.5 GT/s per pin, using standard GDDR6 graphics memory. By supporting a maximum of eight channels in 2x16-bit configuration (which is a maximum of 256 bits), applications can get up to 560 GB/s of low-latency bandwidth to satiate Xe-HPG’s heavy compute needs.

From a memory footprint perspective, Xe-HPG supports from 4 GB to 16 GB of local memory, depending on the SKU.

PCI Express Gen4

Xe-HPG supports a PCI Express* 4.0 host interface, which provides a maximum of 32 GB/s in each direction for a x16 PCIe* 4.0 slot.



Figure 2. X^e-HPG maximum shipping configuration.

X^e-HPG Graphics Architecture

X^e-HPG graphics architecture is the next generation of discrete graphics, adding significant microarchitectural effort to improve performance per-watt efficiency. X^e-HPG's underlying architecture is scalable, so that devices can be constructed to meet a wide range of product performance demands across a wide range of thermal design power.

X^e-HPG has enhancements to support the latest APIs, including DirectX 12 Ultimate features, and corresponding support in Vulkan APIs. As shown in Figure 2, the flagship device features eight render slices with four X^e-cores, each containing 16 XVEs for a total of 512 XVEs, and 16 MB of L2\$, as shown in the following.

New X^e-HPG Render Slice and X^e-core Design

As with X^e-LP, the basic scalable building block of the X^e-HPG compute + render engine architecture is called the "slice." In X^e-HPG, the number of slices varies from 2 to 8, based on the SKU. Increasing the slice count increases overall 3D and compute assets in the machine in a balanced fashion, and can be leveraged to build a wide range of SKUs that support different power and performance needs.

Each slice is built from several X^e-cores, each of which contain 16 vector engines (XVE), 16 matrix engines (XMX), a load/store unit (or data port), and a shared L1/SLM cache. The current instance of the X^e-HPG slice contains a maximum of four X^e-cores.

Each X^e-core is attached to a set of dedicated graphics acceleration fixed function pipelines with a ray-tracing unit (RTU), a thread-sorting unit (TSU), and, of course, a sampler.

In addition, each slice contains the remainder of the fixed function blocks that support rendering functions, including geometry/tessellation, mesh dispatch logic, setup/raster, depth and stencil processing, pixel dispatch logic, and pixel back end. X^e-HPG has increased the fixed function-to-compute ratio, compared to X^e-LP, to address increasing requirements.

X^e-HPG's overall goal was to increase performance-per-watt or efficiency, through improvements in performance at frequency, increased frequency at voltage, and reduced energy per frame.

Performance at frequency was achieved via key architectural changes, as well as compiler optimizations targeting newer workloads. For X^e-HPG, there were significant efforts made to target higher frequency.

Additionally, the N6 process also offers additional v-f improvements. The goal was to achieve up to 1.5x increase in frequency at iso voltage. Energy-per-frame is reduced by micro-architectural optimizations, and intrinsic dynamic capacitance (C_{dyn}) reductions. Reduction in bandwidth utilization through better caching and compression allows redirection of power to the GPU, improving performance.

These improvements allow X^e-HPG to run up to 2.1 GHz in its most performant configuration. This allows X^e-HPG to deliver:

- Up to 17.2 TFLOPs of peak single precision (FP32) performance.
- Up to 137.6 TOPs of peak half precision (FP16) performance using XMX.
- Up to 275.2 TOPs of peak 8bit integer (INT8) performance using XMX.



Figure 3. X^e-HPG render slice.

X^e-core Overview

X^e-HPG brings considerable advances to the X^e-core design, which are realized in gaming and compute workloads. The vector engine (XVE) is the subblock executing instructions, and is similar to the block named execution unit, or EU, in the X^e-LP architecture. In each XVE, the primary computation units are Single Instruction, Multiple Data (SIMD) floating-point units (known as arithmetic logic units, or ALUs). Although called ALUs, they can support floating-point and integer instructions such as MAD or MUL, as well as extended math (EM) instructions, such as exp, log, and rcp. In addition, the ALUs also support logical instructions. On X^e-LP architecture, each EU can co-issue an eight-wide ALU

floating-point or integer instruction, plus a two-wide extended math instruction. X^e-LP also allows Send and Branch to be co-issued with the ALU operations.

From a raw compute perspective, X^e-HPG reorganized the ALUs with the ability to co-issue floating-point instructions and integer instructions on the XVEs. X^e-HPG also adds the X^e Matrix Extension (XMX), a new systolic array of ALUs.

This instruction, named Dot Product Accumulate Systolic (DPAS), can be co-issued as well.

Both architectures derive data-locality benefits from running two XVEs, or two EUs, in lockstep (see Figure 4).

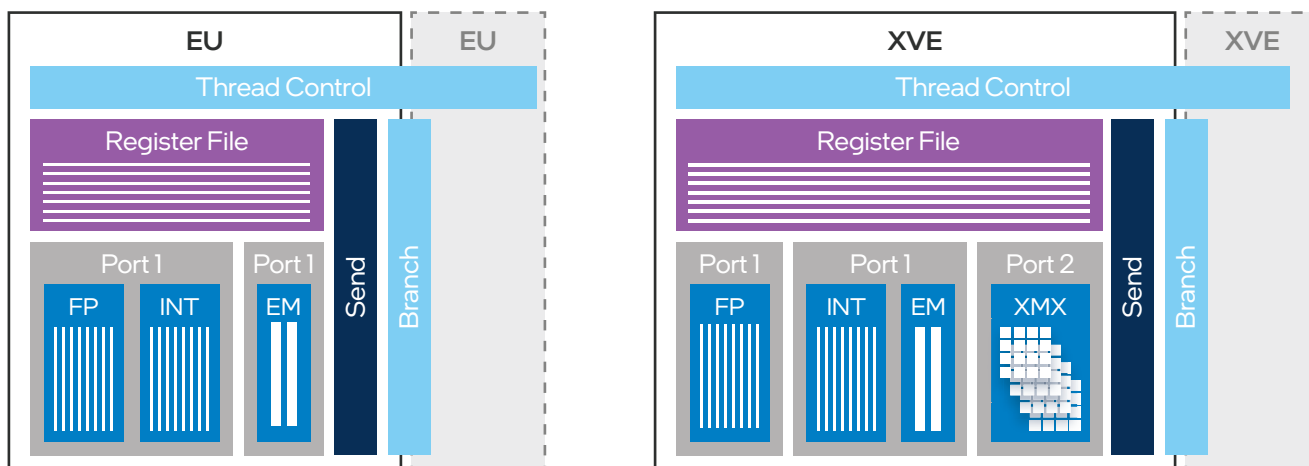


Figure 4. X^e-LP execution unit (EU) versus X^e-HPG vector engine (XVE).

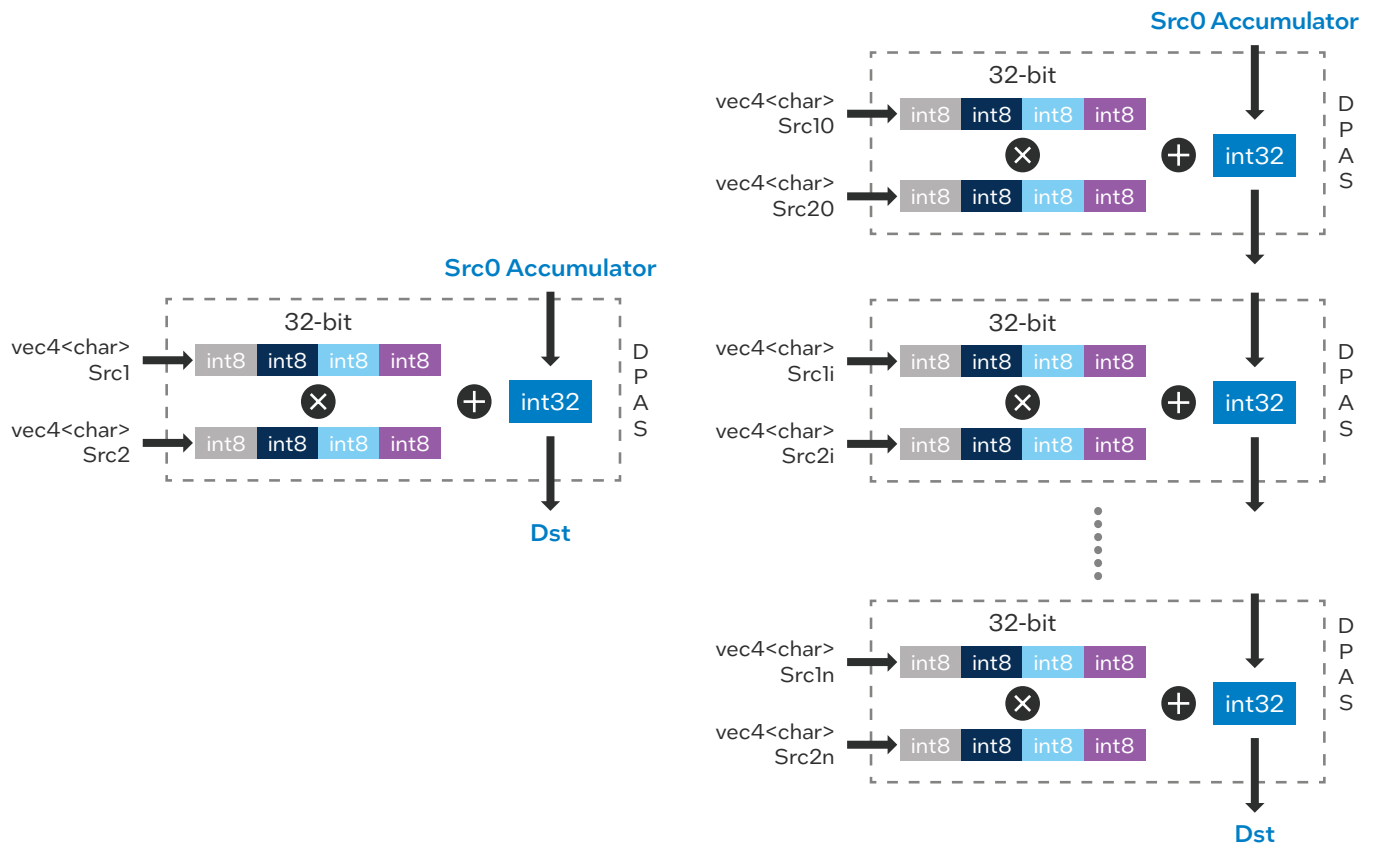


Figure 5. DP4A (X^e-LP and X^e-HPG) and DPAS (X^e-HPG only).

X^e-HPG makes further design changes to improve latencies, as well as improving latency tolerance by adding an additional thread per XVE and corresponding register file (GRF) increase.

Part of the X^e-core is also the instruction cache. X^e-HPG adds a prefetch mechanism to improve the performance of kernels, which are short when instruction fetch constitutes much of the thread execution and displays a high degree of temporal and spatial locality.

Deep learning networks have become one of the biggest general-purpose GPU (GPGPU) usage models, and can use lower precision instructions with minor impact to inference accuracy. As with X^e-LP, X^e-HPG supports INT8 accessed via DP4a instruction. However, one of the changes made to the X^e-core in X^e-HPG, is the addition of 16 XMX engines that can be accessed using the new Dot Product Accumulate Systolic (DPAS) instruction. It supports FP16, BF16, INT8, INT4 and INT2 multiply, with either 16 or 32 bits accumulate. Figure 5 shows both DP4A and DPAS instruction flow.

The throughput rates for the various instructions are shown in Table 1.

Table 1. EU/XVE throughput rates.

Metrics per EU/XVE	X ^e -LP	X ^e -HPG
FP32 FLOPs per Clock (MAD)	16	16
FP16 FLOPs per Clock (MAD)	32	32
INT32 Ops per Clock (ADD)	8	8
INT16 Ops per Clock (MAD)	32	32
INT8 Ops per Clock (DP4A)	64	64
XMX FP16 Ops per Clock (DPAS)	–	128
XMX BF16 Ops per Clock (DPAS)	–	128
XMX INT8 Ops per Clock (DPAS)	–	256
XMX INT4/INT2 Ops per Clock (DPAS)	–	512
Extended Math Ops per Clock (EXP, RCP)	2	2
Thread Count	7	8
Register File	28KB	32KB

Xe-core Fixed Function

The texture sampler is a critical unit attached to the Xe-core that supports up to a peak of eight texels per clock, similar to Xe-LP. Intel improved the sampler capability to run at higher frequencies, but there are also other performance optimizations. One of the notable changes is improved anisotropic performance for some of the block compression (BCx) formats, which are common in games. Other improvements enhance power efficiency as well as performance for certain formats commonly used in games.

To include state-of-the-art lighting and visualization, each of the Xe-cores contains a ray-tracing unit (RTU). RTUs process ray-tracing related messages from the XVEs to kick off traversal including intersection tests. Each RTU has a bounding volume hierarchy (BVH) cache to reduce the average latency of fetching BVH data and is capable of processing multiple rays for higher efficiency. Each RTU supports a 12-to-1 ratio of ray-box intersection tests per clock and ray-triangle tests per clock.

Additionally, each RTU is supported by a thread-sorting unit (TSU), a dedicated hardware block that can sort and re-emit shader threads to maximize SIMD coherence from divergent rays.

Xe-core L1\$

To meet the high bandwidth demand of the XMX instructions and the ray-tracing units, Xe-HPG implements a newly designed read/write L1-cache unit in each Xe-core. The new L1-cache unit handles all unformatted load/store accesses to memory, and shared local memory (SLM). Based on the shader requirement, the cache storage can be dynamically partitioned between L1-cache and SLM, featuring up to 192 KB of L1 cache or up to 128 KB of SLM. Formatted data load/

store (such as typed unordered access views [UAVs]) are handled by the data port pipeline, which shares a separate, 64 KB read-only cache with the texture sampler unit.

In addition to improving the performance of ML/DL workloads and ray tracing, the low-latency, high-bandwidth L1 cache also improves shaders with frequent accesses to SLM, dynamically indexed constant buffers, and high register spill-fills.

Xe-HPG Slice Common

The Xe-HPG geometry fixed function pipeline contains the typical render front end that maps to the logical API pipeline. Enhancements include provisions to support distribution of the geometry work between the various slices for larger devices, while increasing peak fixed-function rate.

A significant change is the addition of a two-stage mesh-shading pipeline introduced in DirectX Ultimate as well as Vulkan.

Although mesh-shading functionality appears to software as a separate pipeline feeding into the rasterization function, the implementation is an alternate operating mode of the geometry fixed function, leveraging similar existing device assets. When operating in mesh-shading mode:

- The global vertex fetch distributes geometry work to slices when in geometry mode and is reused to distribute mesh-shader dispatch work to slices in a load-balanced fashion.
- The Hull Shader (HS) stage in each slice geometry pipeline alternatively supports the optional Amplification Shader (AS) stage of the mesh-shader compute pipeline.

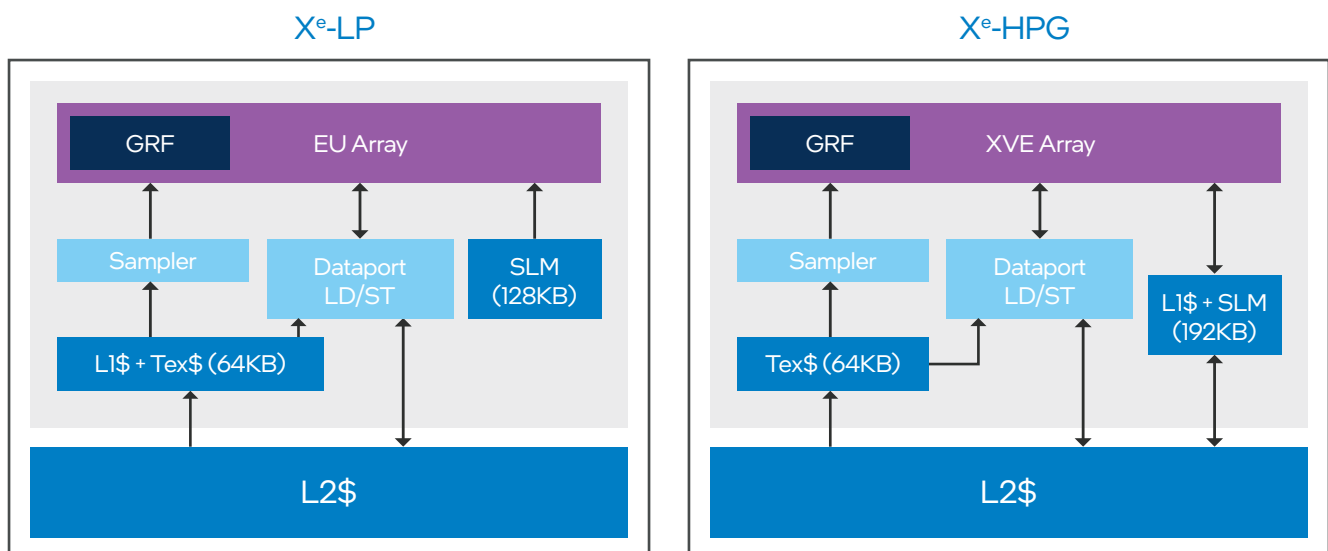


Figure 6. New shared local memory (SLM) and L1\$.

- In geometry mode, the tessellation (TE) stage logic in each slice geometry pipeline, and the tessellation redistribution (TED) logic in the shared functions, are used to redistribute tessellation work (produced by the slices' HS stages) across the lower portions of the slices' geometry pipelines. In mesh shading mode, this logic is reused to redistribute and load-balance mesh-shading thread-group work across the geometry pipelines.
- The geometry shader (GS) stage in each slice geometry pipeline alternatively supports the mesh shader (MS) stage of the mesh-shader compute pipeline. Outputs of the GS (in either geometry or mesh-shading mode) are passed down to the rasterization pipeline.

To support the rendering of mesh-shading output, the Xe rasterization pipeline is extended to support per-primitive pixel-shader attributes, in addition to the existing per-vertex attributes. Peak rates for mesh-shading function are provided in Table 2.

Table 2. Key peak metrics for mesh shading.

Key Peak Metrics	Clocks/Dispatch/Slice
Amplification Shader (AS) Null Output	2
Mesh Shader (MS) only Null Output	1
Amplification Shader (AS) MeshShader Launch Count = 1	9

Additionally, the slice common also includes the pixel dispatch unit, which accumulates subspans/pixel information and dispatches threads to the XVEs. The block load balances across the multiple Xe-cores and ensures the order in which pixels retire from the shader units. It also includes support for variable rate shading (VRS), which reduces the pixel shader invocations, and in Xe-HPG has been enhanced from Tier 1 to Tier 2 support.

Depth tests are performed at two levels of granularity: coarse and fine. The coarse tests are performed by hierarchical depth (HiZ) where testing is done on 8x4 pixel block granularity. In addition, the HiZ block supports fast clear, which allows clearing depth without writing the depth buffer.

The pixel back end (PBE) is the last stage of the rendering pipeline, which includes the cache to hold the color values backed by the L2 cache. This pipeline stage also handles the color blend functions across several source and destination surface formats. Each PBE supports up to 8 pixels/clock for alpha-blended surfaces for a total of 16 pixels/clock per render slice.

L2 Cache, Compression, and Memory

L2 cache is the last level cache in the memory hierarchy. Memory requests from all the render slices and Xe-cores are directed to the L2 cache. As such, it is the largest unified storage structure within the IP, and serves as the final filter for data moving to and from device memory. The L2 cache is used as backing storage for sampler, data port, color, Z, UAV, and instruction streams.

The L2 cache is a highly banked multiway set-associative cache. Fine-grain controls in the L2 allow selective caching of data, as well as provisioning of different classes of services to streams of traffic. Each of the banks can perform one 64 byte read-or-write operation every clock. The L2 cache forms a single contiguous memory space across all the banks and sub-banks in the design. The 32 Xe-cores configuration supports a peak of 2048 bytes/clock for read or write. In typical 3D/compute workloads, partial access is common and occurs in batches, making ineffective use of memory bandwidth. L2 cache opportunistically combines partial accesses into a single 64Byte access, improving efficiency.

Xe-HPG graphics architecture continues the investment in technologies that improve graphic memory efficiency, in addition to improving raw memory bandwidth. As games target higher quality visuals, the memory bandwidth requirement significantly increases. Xe-HPG uses a new unified, lossless compression algorithm that is universal across color, depth, and stencil, as well as media and compute. The data can be stored in the L2 cache in compressed as well as uncompressed form, yielding benefit for both capacity and bandwidth.

Global Assets

Global Assets presents a hardware and software interface to and from the GPU, including power management.

Specifications Tables

Peak Rates

Table 3, below, presents the theoretical peak throughput of the compute architecture of Intel® processor graphics, aggregated across the entire graphics product architecture. Values are stated as “per clock cycle”.

Table 3. The theoretical peak throughput of Xe-HPG and Xe-LP.

Key Peak Metrics	Xe-LP	Xe-HPG [ACM-G11]	Xe-HPG [ACM-G10]
# of Xe-cores	-	8	32
# of Slices	1	2	8
# of Xe-cores / Slice	6 subslices	4	4
# of XVEs/EUs (Total)	96 (1x6x16)	128 (2x4x16)	512 (8x4x16)
FP32 (SP) FLOPs/Clock	1536	2048	8192
FP16 (SP) FLOPs/Clock	3072	4096	16384
INT32 IOPs/Clock (ADD)	768	1024	4096
INT8 IOPs/Clock (DP4a)	6144	8192	32768
XMV FP16 Ops per Clock (DPAS)	–	16384	65536
XMV BF16 Ops per Clock (DPAS)	–	16384	65536
XMV INT8 Ops per Clock (DPAS)	–	32768	131072
XMV INT4/INT2 Ops per Clock (DPAS)	–	65536	262144
General Register File/XVE (KB)	28	32	32
Total Register File (KB)	2688	4096	16384
# of Samplers	6	8	32
Point/Bilinear Texel's/Clock (32 bpt)	48	64	256
Point/Bilinear Texel's/Clock (64 bpt)	48	64	256
Trilinear Texel's /Clock (32bpt)	24	32	128
Ray-Box Intersect/Clock	–	96	384
Ray-Triangle Intersect/Clock	–	8	32
L1\$ Total (KB)	384	1536	6144
Shared Local Memory Total (KB)	768	1024	4096

Table 3. Continued

Key Peak Metrics	Xe-LP [DG1]	Xe-HPG [ACM-G11]	Xe-HPG [ACM-G10]
Slice Common & Geometry Attributes			
Pixel Fill – RGBA8 (Pixels/Clock)	24	32	128
Pixel Fill w. Alpha Blend - RGBA8 (Pixels/Clock)	24	32	128
HiZ Pixels/Clock	256	512	2048
Backface Cull – strips (Prim/Clock)	2	4	16
Backface Cull – lists (Prim/Clock)	0.67	2.66	10.64
L2\$ Cache (KB)	16384	4096	16384
Memory Subsystem Attributes			
Memory Configuration (bits)	128	96	256
Max Memory Data Rate (GT/s)	4.267	15.5	17.5
Max Frame Buffer Size (GB)	4	6	16
Memory Bandwidth (GB/s)	68	186	560

Graphics API Support

Xe-HPG supports all the major APIs: DirectX, OpenGL*, Vulkan, and OpenCL™. Table 4 below shows the features discussed above, and others that are part of Direct 3D 12 and Vulkan, mapped to Xe-LP and Xe-HPG.

Table 4. D3D12 and Vulkan APIs support for Xe-LP and Xe-HPG.

API Support	DirectX 12		Vulkan*	
	Xe-LP	Xe-HPG	Xe-LP	Xe-HPG
Max Feature Level	12_1	12_2	–	–
Shader Model	6_6	6_6	–	–
Resource Binding	Tier 3	Tier 3	Based on Driver Query	Based on Driver Query
Tiled Resources	Tier 3	Tier 3	Based on Driver Query	Based on Driver Query
Typed UAV Loads	Yes	Yes	Yes	Yes
Conservative Rasterization	Tier 3	Tier 3	Yes – VK_KHR_conservative_rasterization	Yes – VK_KHR_conservative_rasterization
Rasterizer-Ordered Views	Yes	Yes	Yes – VK_EXT_fragment_shader_interlock	Yes – VK_EXT_fragment_shader_interlock
Stencil Reference Output	Yes	Yes	VK_EXT_shader_stencil_export	VK_EXT_shader_stencil_export
UAV Slots	Full Heap	Full Heap	Based on Driver Query	Based on Driver Query
Resource Heap	Tier 2	Tier 2	–	–
Variable Rate Shading	Tier 1	Tier 2	Yes – VK_KHR_fragment_shading_rate	Yes – VK_KHR_fragment_shading_rate
View Instancing	Tier 2	Tier 2	Yes - VK_KHR_multiview	Yes - VK_KHR_multiview
Asynchronous Compute	Yes	Yes	Yes	Yes
Depth Bounds Test	Yes	Yes	Yes	Yes
Sampler Feedback	Tier 0.9	Tier 0.9	TBD	TBD
Ray Tracing	No	Tier 1.1	No	Yes – extensions
Mesh Shading	No	Tier 1.0	No	Yes – extensions

Conclusion

The Xe-HPG architecture provides scalability and a modern DirectX 12 Ultimate graphics feature set to power the Intel Arc A-series product line. The following guides are to help you go further and fully use the Xe-HPG architecture for your graphics applications.

Resources

- Intel® Xe Super Sampling (XeSS) API Developer Guide
- Intel® Arc™ A-series Graphics Gaming API Developer and Optimization Guide
- Intel® Arc™ Graphics Developer Guide for Real-Time Ray Tracing in Games
- Intel OpenVINO™ API
- Microsoft* WinML API
- Intel-optimized CLDNN kernels
- The Coalition Studio



¹ Assumes Xe-LP and Xe-HPG at fmax for each product.

Intel technologies' features and benefits depend on system configuration and may require enabled hardware, software, or service activation. Performance varies depending on system configuration. Check with your system manufacturer or retailer or learn more at [intel.com](https://www.intel.com).

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

This document contains information on products, services, and/or processes in development. All information provided here is subject to change without notice. Contact your Intel representative to obtain the latest forecast, schedule, specifications, and roadmaps.

The products and services described may contain defects or errors known as errata which may cause deviations from published specifications. Current characterized errata are available on request.

Copies of documents which have an order number and are referenced in this document may be obtained by calling 1-800-548-4725 or by visiting www.intel.com/design/literature.htm.

Arc, Intel, the Intel logo, and OpenVINO are trademarks of Intel Corporation or its subsidiaries.

* Other names and brands may be claimed as the property of others.

Microsoft, Windows, and the Windows logo are trademarks, or registered trademarks of Microsoft Corporation in the United States and/or other countries.

OpenCL and the OpenCL logo are trademarks of Apple Inc., used by permission by Khronos.